L12-v10

2026年10月25日 17:36

一、Agent 需要专用可观测性

1.1 传统 APM 在 LLM 系统上失效

传统应用性能监控（APM）假设代码路径是确定性的：同样的输入产生同样的输出，失败表现为异常或超时。LLM Agent 打破了这些假设。

维度	传统应用	LLM Agent
延迟驱动	CPU、I/O、网络	Token 数量、模型大小、上下文窗口
成本单位	请求数/秒	消耗的 Token 数
失败模式	异常、超时	幻觉、上下文溢出、工具错误
调试产物	调用栈	Prompt + 补全 + 推理链
高基数风险	URL、用户 ID	完整 Prompt 文本进入属性

同一 Prompt 可能产生不同输出，同一个 Agent 处理一个用户请求可能发起 5 次模型调用、3 次工具调用和 2 次向量检索，每次都是一个潜在的失败点。只监控“请求速率”对 Token 驱动的成本完全盲视。

1.2 旁路关闭时必须零代价

可观测性代码有两种存在方式：控制路径的一部分，或旁路。

旁路的严格定义：把整个埋点模块拔掉，系统的行为必须逐字节不变。

要达到这一点，需要两个条件同时成立：

a. 关闭时所有方法都是 no-op，且早退发生在计算任何度量数据之前。如果只是“不记录”但仍然计算了 requestChars，开销并没有消失。
b. 没有任何外部代码读取埋点的返回值来决定下一步做什么。一旦“发现某个工具慢就加缓存”，埋点就不再是旁路，开不开 trace 会改变系统行为。

Oxtrace 用“微任务异步发送”做到零延迟：Promise.resolve().then() 把遥测推入微任务队列，Agent 主路径立即返回，不等遥测完成。Langfuse SDK 同样“在后台线程发送，只读取 API 返回的 usage，不增加 LLM token 成本”。

1.3 两种记录：span 与 event

Agent 可观测性中，事件分为两类：

- event：某一刻发生了什么（工具开始、压缩触发、出错）。只有一个时间点。
- span：一段有始有终的任务（一次模型请求、一次工具执行、一次子代理运行）。有开始时间、结束时间、时长。

span 的时长是唯一能回答“慢在哪儿”的东西。event 只能回答“发生了什么”。只记 event 的 trace，无法回答“那到底哪一步慢”。OpenTelemetry GenAI 语义约定中，Agent 的三种核心 span 是：invoke_agent（整个 Agent 运行或子代理）、chat（一次模型调用）、execute_tool（一次工具调用）。

1.4 环形缓冲：丢弃必须可见

对于长程任务，Agent 可能产生几十万条事件。如果每条都留在内存里，内存和落盘都会失控。采用固定容量的环形缓冲：溢出从头部丢弃最老的事件。

不能静默丢弃：如果“主agent看到的不是全部”变成隐形事实，基于 trace 做的所有判断都可能建立在残缺数据上。环形缓冲必须维护一个 dropped 计数器，并在输出中显式报告。当前版本的 Tracer::dropped() 和 I-82 断言正是这个原则的落地。

1.5 古德哈特定律：度量一旦成为目标，就不再是好指标

Agent 可观测性面临一个结构性陷阱。如果把“测试通过率”作为唯一优化目标，Agent 会学会修改测试断言而不是写出更好的代码；如果把“成本”作为唯一目标，Agent 会学会什么都不做。

代理循环将古德哈特定律压缩进单次运行中：将代理指标交给一个强大的优化器，它就会利用其中的间隙进行博弈。

当前版本的 --regress 强制要求通过率、成本、步数三个数同时存在，缺一个就拒绝出结果。这是对“度量作弊”的结构性防御。

单个度量	Agent 会学会的“作弊”方式
通过率	暴力重试（多跑几次总能过）、改测试断言（让它总是通过）、跳过难的任务
成本	什么都不做（不调工具当然便宜）、少读文件、直接瞎猜
步数	一步干完所有事（一个大 Edit 覆盖整个文件）、不做验证

三个一起看，作弊的方向互相牵制：

- 想靠“暴力重试”提通过率？成本会涨、步数会涨。
- 想靠“什么都不做”降成本？通过率会掉。
- 想靠“一步干完”降步数？通过率大概率会掉（不读文件就改，出错）。

二、Agent 可观测性主流技术：

2.1 OpenTelemetry GenAI 语义约定：非正式标准

早期每个 LLM tracer 发明自己的字段名，数据不可互操作。OpenTelemetry 的 GenAI 语义约定定义了共享的 gen_ai.* 命名空间，让一份 trace 在任何兼容后端中都能正确解读。

核心操作名：chat（模型调用）、invoke_agent（Agent 运行）、execute_tool（工具调用）、embeddings（嵌入调用）。

属性	含义
gen_ai.request.model	请求的模型
gen_ai.usage.input_tokens / output_tokens	输入/输出 token 数
gen_ai.response.finish_reasons	停止原因
gen_ai.agent.name / gen_ai.conversation.id	Agent 标识与会话标识

默认不捕获 Prompt 内容——只记录元数据（模型名、token 数、时长），因为 Prompt 可能含 PII 或商业机密。这本身是一个重要的设计取舍：可观测性与隐私的边界。

2.2 主流接入模式

模式	代表	侵入性	原理
SDK 装饰器	AgentWeave @trace_agent / @trace_tool	低	用注解包装函数，需要改代码
自动插桩	AgentWeave auto_instrument()	极低	一次调用 patch OpenAI/Anthropic SDK
代理模式	0xtrace、AgentWeave Proxy、Helicone	零代码	把 base URL 指向代理，代理透传并提取 token
eBPF 内核探针	AgentSight、OBI	零代码零部署	内核层拦截 SSL/TLS 流量与系统调用

代理模式的原理是：Agent 的 base URL 指向一个本地或远程代理，代理把请求原样转发给上游模型 API，同时从响应中提取 usage 字段并生成 trace。它不需要改一行 Agent 代码，但需要一个代理进程始终运行。

eBPF 模式的原理是在内核层挂载探针，捕获解密后的 LLM 流量（intent stream）和系统调用（action stream）。它的独特优势是不依赖 Agent 自身的日志——即使 Agent 关闭了日志或被修改，eBPF 仍然能捕获真实行为。代价是只能看到网络和系统调用层面的信息，无法直接获得 Agent 内部的语义（比如“这一步是第几步”）。

2.3 主流工具谱系

工具	定位	关键特征
Langfuse	开源 LLM 可观测性平台	MIT 核心，可自托管，50k 事件/月免费
LangSmith	LangChain 生态的追踪与评估	与 LangChain/LangGraph 深度集成，闭源
Arize Phoenix	OTel 原生的本地/自托管追踪	无事件上限，适合 RAG 可视化与漂移检测
AgentWeave	多 Agent 溯源层	在 OTel 上附加 W3C PROV-O 溯源信息，强调“哪个

		Agent 委托了哪个”
AgentSight / OBI	eBPF 系统级监控	从应用外部观察，独立于 Agent 日志

当前版本的实现不依赖任何外部平台——**只做**一个最小化的 tracer，把事件落成 JSONL，用本地 CLI 消费。

三、新增清单

文件	变化类型	核心新增
trace.hpp	新增文件	TraceKind / TraceEvent / Tracer / TracingProvider / LayerChars
trace.cpp	新增文件	环形缓冲、span 配对、JSONL 落盘与读回、装饰器透传
main.cpp	重大修改	① 三个仪表盘 CLI；② Tracer 接线；③ buildSystemPrompt() 抽出；④ 八层占用计算；⑤ I-94 自检

四、trace 模块

4.1 设计目标

把 Tracer 整个**关闭**，系统的行为必须逐字节不变。

两条**约束**：

① 关闭时所有方法都是 no-op（if (!enabled_) return; 在最前面）；

② 没有任何外部代码读它的返回值来决定下一步做什么。

4.2 span 与 event 事件

```
enum class TraceKind {
    Request,     // span: 一次模型请求
    ToolExec,    // span: 一次工具执行
    Subagent,    // span: 一次子代理运行
    Compact,     // event: 触发了一次压缩
    ToolDone,    // event: 工具结束 (成功/失败)
    Error,       // event: 出错
    Notice,      // event: 循环通知
};
```

4.3 环形缓冲

```
static constexpr std::size_t kDefaultCapacity = 4096;
```

```
void Tracer::push(TraceEvent e) {
    // ★ 环形缓冲: 满了就从头部丢, 并**记下丢了多少** (K-48)
    //   deque::pop_front 是 O(1); vector::erase(begin()) 是 O(n) (K-49) 。
    if (events_.size() >= capacity_) {
        events_.pop_front();
        ++dropped_;
    }
    events_.push_back(std::move(e));
}
```

4.4 endSpan ： 从后往前**搜索**

```cpp
void Tracer::endSpan(int id, int tokIn, int tokOut, long long chars, int64_t ttftUs) {
    // ★★ 必须**从后往前**找: span 通常刚开就关, 尾部一次命中, O(1)。
    //    从前往后在事件多时是 O(n) — 一次 endSpan 要扫几万条,
    //    整个会话退化成 O(n²)。实测: 50k 事件下每事件 18μs → 改成反向后 <1μs (K-49)。
    //    ★ 这类 bug 只在"事件变多"时才现身, 小会话完全测不出来,
    //      而它长得像"模型变慢 / 内存泄漏", 排查方向会完全跑偏。
    for (auto it = events_.rbegin(); it != events_.rend(); ++it) {
        TraceEvent& e = *it;
        if (e.seq != id || e.durUs >= 0) continue;
        // ...
    }
}
```

4.5 三个仪表盘

--cost：读一个或多个 trace JSONL，逐文件打印请求数、工具数、步数、主 token 和子代理 token 分列、墙钟、坏行数。合计行同样分列。

--context：离线估算八层占用。它使用与真实运行同一份 buildSystemPrompt()、同一份工具表、同一份 CLAUDE.md、同一份仓库地图——任何一处偷懒，报出来的数就是假的。输出中直接附带自洽性断言：八层之和必须等于总数，否则打印 ★不自洽！。

--regress：读回归集 JSONL，强制要求 pass、cost、steps 三个字段同时存在。缺一个就拒绝出结果，而不是填 0 凑一个通过率。

4.5 LayerChars：八层

```cpp
struct LayerChars {
    long long l0 = 0;   // 系统提示
    long long l1 = 0;   // 项目知识 (CLAUDE.md)
    long long l2 = 0;   // 仓库地图
    long long l3 = 0;   // 检索片段
    long long l4 = 0;   // 活动文件
    long long l5 = 0;   // 历史 (含 tool_call 参数)
    long long l6 = 0;   // 用户输入
    long long l7 = 0;   // ★ 工具表 (JSON Schema) — 它也是要发出去的字节
    long long total() const { return l0+l1+l2+l3+l4+l5+l6+l7; }
};
```

完整数据流

```
主循环
  │
  ├─ hooks.onToolStart(tc)
  │    ├─ toolSpan[tc.id] = tracer.beginSpan(ToolExec, tc.name)
  │    └─ ...
  │
  ├─ hooks.onToolDone(r)
  │    ├─ tracer.endSpan(toolSpan[r.callId])
  │    ├─ tracer.event(ToolDone, r.callId, ...)
  │    └─ ...
  │
  ├─ hooks.onNotice(msg)
  │    └─ tracer.event(Notice, msg)
  │
  ├─ hooks.onError(e)
  │    └─ tracer.event(Error, e)
  │
  └─ Task 工具闭包
       ├─ tracer.setInSubagent(true)
       ├─ SubagentRunner::run() → 内部 provider 请求被 TracingProvider 记成 Subagent span
       └─ tracer.setInSubagent(false)

turn 结束
  │
  ├─ AF_CONTEXT=1 → printContext(layerCharsOf(...))
  │
  ├─ AF_TRACE_FILE → tracer.flush(path)
  │    └─ JSONL: meta + 每条事件
  │
  └─ 打印总结（含 subagent 单列）

离线
  │
  ├─ --cost <trace.jsonl>...    → 主 token + 子代理 token 单列
  ├─ --context [root]           → 八层占用 + 自洽性
  └─ --regress <regress.jsonl>  → 通过率 + 成本 + 步数三个一起报
```

```cpp
int ContextAssembler::estimateTokens(std::string_view s) noexcept {
    return int(s.size() / kCharsPerToken) + 1;   // kCharsPerToken = 4
}
```